Guide du developpeur d'agent

Comment construire, tester et soumettre votre agent

Ce guide s'adresse a chaque equipe etudiante, quelle que soit sa filiere. Il part du principe que vous avez deja lu AGENT_INTEGRATION_CONTRACT.docx et, si votre agent echange des donnees avec un autre agent, INTER_AGENT_DATA_CONTRACT.docx.

Etape 0 — Partir de l'agent exemple

Le noyau est livre avec un dossier agent-exemple/, qui contient un agent minimal mais complet : les 5 endpoints obligatoires, un endpoint exchange/notify de demonstration, un manifeste, un Dockerfile. Copiez ce dossier pour demarrer votre propre agent plutot que de repartir de zero.

```
cp -r agent-exemple mon-agent
cd mon-agent
```

Etape 1 — Choisir votre stack technique

Même si noyau n'impose aucun langage : Python, Node.js, Go, Java... du moment que votre agent expose une API HTTP/JSON conforme au contrat. Cependant, pour une meilleure cohérence du projet, le langage Python doit être utiliser. Par ailleurs, l'agent exemple est ecrit en Python/FastAPI ce qui est rapide a prendre en main.

Etape 2 — Adapter le manifeste

Modifiez agent.manifest.json : slug, name, description, filiere, category, developer, team_name. Laissez produces et consumes vides si votre agent n'echange pas de donnees avec un autre agent.

Etape 3 — Developper votre logique metier

Remplacez le code de demonstration de main.py par le comportement reel de votre agent, en respectant les regles de base :

- Filtrez toujours vos donnees par tenant_id.
- N'appelez jamais directement l'adresse d'un autre agent : passez par le bus d'echange du noyau si necessaire.
- Ne loggez jamais de donnees sensibles en clair.

Etape 4 — Envoyer une alerte (si besoin)

```
curl -X POST http://localhost:8000/webhooks/agent-events \
  -H "Authorization: Bearer <tenant_api_key>" \
  -H "X-Tenant-Id: <company_id>" \
  -H "Content-Type: application/json" \
  -d '{"event_type": "seuil_depasse", "severity": "medium", "data": {"valeur": 95}}'
```

Etape 5 — Publier et recevoir des objets (selon le besoin)

Si votre agent produit un objet :

```
curl -X POST http://localhost:8000/exchange/agent/stock_level \
  -H "Authorization: Bearer <tenant_api_key>" \
  -H "X-Tenant-Id: <company_id>" \
  -d '{"external_id": "SKU-001", "payload": {"quantite_disponible": 37}}'
```

Si votre agent consomme un objet, implementez /exchange/notify (deja present dans l'agent exemple) : le noyau l'appellera automatiquement des qu'un objet du type concerne est publie par un autre agent, ou par un humain depuis le tableau de bord.

Etape 6 — Conteneuriser

Adaptez le Dockerfile fourni si votre stack differe de Python. Verifiez que l'image demarre sans configuration manuelle autre que les variables d'environnement.

Etape 7 — Tester en local avant de soumettre

Lancez le noyau et votre agent en parallele, puis simulez vous-meme : inscription d'une entreprise de test (page /inscription), souscription a votre agent une fois publie (voir etape 9), activation, et si applicable, publication/reception d'un objet de test.

Etape 8 — Soumettre votre agent

```
curl -X POST http://localhost:8000/agents/submit \
  -H "Content-Type: application/json" \
  -d @agent.manifest.json
```

Votre agent passe en statut "soumis". L'enseignant le verifie depuis la page /admin.

Etape 9 — Certification et publication

Une fois verifie conforme, l'enseignant certifie puis publie votre agent depuis /admin. Il apparait alors dans le catalogue (/catalogue) et devient souscriptible par une entreprise de test ou de demonstration.

Liste de verification avant de soumettre

- Les 5 endpoints obligatoires repondent avec l'enveloppe standard {status, data, error}.
- Le manifeste est valide, le slug unique, la filiere renseignee.
- Isolation par tenant_id verifiee manuellement (deux entreprises de test ne se voient jamais).
- Le Dockerfile fonctionne (docker build puis docker run).
- POST /tenant/deactivate supprime reellement les donnees de test.
- Si consumes n'est pas vide : /exchange/notify a ete teste avec un objet factice.